IA Agêntica no Android

https://gemini.google.com/app/e65a107509017fab

User prompt: tá e sobre o hermes agent no celular via iinterface web local?

Response: Implementar uma interface web local para o Hermes Agent no celular é a rota mais eficiente. Com 12 GB de RAM LPDDR5X, o dispositivo tem fôlego para manter o contêiner Debian, um modelo quantizado (como o Hermes 3 8B ou Qwen 2.5 Coder 7B), o orquestrador agêntico e o servidor web rodando simultaneamente em segundo plano.

Para que a interface funcione no navegador do Android se comportando como um assistente completo, a arquitetura precisa ser dividida em camadas dentro do seu ambiente proot.

1. **Iniciar o LLM Local**

   Ollama / llama.cpp

   Dentro do seu contêiner Ubuntu/Debian no Termux, o primeiro passo é subir a API do modelo que servirá de "cérebro" para o agente. Inicie o servidor do Ollama em background para que o Hermes Agent possa enviar os prompts e receber as chamadas de função (function calling). *Verification:* O comando `curl [http://127.0.0.1:11434/api/tags](http://127.0.0.1:11434/api/tags)` retorna a lista dos seus modelos instalados.

2. **Configurar o Servidor Web e o Agente**

   Python / FastAPI ou Chainlit

   O Hermes Agent não possui uma interface gráfica (GUI) nativa embutida. Você precisará de um framework Python para expor o agente.

   - **Para prototipagem rápida:** Use o **Chainlit** ou **Streamlit**. Eles geram uma interface de chat pronta com poucas linhas de código.
   - **Para a arquitetura definitiva:** Use o **FastAPI** como backend, criando endpoints REST/WebSocket que se comunicam com o Hermes Agent e com servidores MCP. O FastAPI servirá os arquivos estáticos (HTML/JS/CSS) de uma interface de chat web responsiva.

3. **Expor e Acessar a Interface**

   Navegador Mobile

   Execute o script do seu servidor web no terminal (ex: `uvicorn app:app --host 0.0.0.0 --port 8000`). Em seguida, minimize o Termux (mantenha-o rodando em segundo plano), abra o Chrome ou Firefox no Android e acesse `[http://127.0.0.1:8000](http://127.0.0.1:8000)`. *Verification:* A página web carrega a interface de chat e responde de forma coerente ao seu primeiro prompt de teste.

A partir dessa fundação, a interface web acessada pelo navegador terá permissão para ler, editar e organizar os arquivos que estiverem acessíveis dentro do sistema de arquivos do Termux.

Se o objetivo final é evoluir dessa interface web para um aplicativo mobile customizado completo (com histórico de múltiplos chats, edição de markdown e gerenciamento visual de arquivos), o FastAPI construído no passo 2 já servirá como o motor ideal. Basta substituir o frontend web estático pela interface do seu próprio app consumindo esses mesmos endpoints.

User prompt: no caso penso em conectar llms de alta capacidade via api gratis, usando um nine router em backeground ou integrado.

Response: Transferir o processamento para LLMs de alta capacidade na nuvem via 9Router (também conhecido como NymRouter) desocupa os 12 GB de RAM do seu Poco X7 Pro e preserva a bateria durante execuções contínuas do agente.

O 9Router opera localmente no seu dispositivo como um proxy de orquestração com um endpoint compatível com a estrutura da OpenAI. A função mais crítica dele ao lidar com APIs gratuitas é o fallback automático: se um provedor rejeitar a conexão ou impor um limite de taxa (rate limit) no meio de um processamento, o router repassa a requisição imediatamente para o próximo provedor da sua lista, impedindo que o Hermes Agent trave ou perca o contexto da operação.

1. **Instalar e Executar o 9Router**

   Ambiente proot

   Instale o 9Router diretamente no seu contêiner Debian/Ubuntu no Termux. Ele funciona como uma aplicação local isolada e salva as configurações em um banco de dados SQLite sem exigir a instalação de serviços adicionais como o Redis. Mantenha-o rodando em segundo plano. *Verification:* Acesse `[http://127.0.0.1:20128](http://127.0.0.1:20128)` no navegador do Android; o painel de controle (dashboard) do 9Router será carregado e solicitará a criação de um login.

2. **Configurar APIs Gratuitas e Fallback**

   Dashboard

   No painel web, cadastre as suas chaves (API Keys) dos provedores de acesso gratuito (opções comuns recomendadas pela comunidade incluem o OpenCode, o NVIDIA NIM, ou o AgentRouter, que fornece acesso a modelos como o DeepSeek e GLM). Em seguida, agrupe essas chaves em uma única rota, definindo a ordem de prioridade para o fallback automático. *Verification:* Na área de rotas do dashboard, execute o teste de ping/conexão; o sistema deve retornar um status "200 OK" para a cadeia de provedores configurada.

3. **Redirecionar o Hermes Agent**

   Backend Python

   No código do servidor FastAPI que controla o agente, troque a URL base que apontava para o seu modelo local no Ollama (`[http://127.0.0.1:11434/v1](http://127.0.0.1:11434/v1)`) pelo endpoint padronizado do 9Router: `[http://127.0.0.1:20128/v1](http://127.0.0.1:20128/v1)`. Altere também a variável de modelo para o nome exato configurado na sua rota do 9Router. *Verification:* Ao enviar um comando de teste pela sua interface web local, o dashboard do 9Router registrará o tráfego e o consumo de tokens, e o agente responderá usando o modelo hospedado na nuvem.

Sources

1. 9Router — One API for Every AI Model
2. 9Router: Free Unlimited AI Coding (2026 Setup) - AI Profit Boardroom
3. Deploy - 9Router — Self-Hosted OpenRouter Alternative - Railway
4. How To Use 9Router For AI Automation (2026)

User prompt: não queria usar o termux, queria usar tudo num "app" só.

Response: Para consolidar tudo em um único aplicativo nativo (.apk) e eliminar a necessidade de gerenciar um ambiente de terminal em segundo plano, a principal barreira é o *Scoped Storage* do Android. O sistema operacional proíbe que um aplicativo padrão tenha acesso autônomo e irrestrito para ler, escrever e organizar todas as pastas do dispositivo.

Atualmente, não há um aplicativo comercial pronto que entregue essa experiência agêntica global no Android. No entanto, existem duas abordagens práticas para alcançar esse resultado em um único app.

1. A Abordagem de "Workspace" Fechado (Obsidian)

Se o foco principal for a escrita, pesquisa e organização de conhecimento, o aplicativo **Obsidian** atua como o ambiente mais próximo de um assistente agêntico nativo, operando dentro de um diretório isolado (o Vault).

- **Autonomia Controlada:** O Obsidian tem acesso total de leitura e escrita dentro do Vault escolhido no seu armazenamento interno.
- **Integração Agêntica:** Utilizando plugins da comunidade (como *Text Generator*, *Copilot* ou *Smart Connections*), você conecta o app diretamente ao seu 9Router ou a APIs externas.
- **Capacidades:** A IA consegue ler todo o contexto dos seus arquivos Markdown, gerar novos documentos, reestruturar notas e realizar pesquisas interligadas sem sair do aplicativo e sem precisar do Termux.

2. O Desenvolvimento de um APK Unificado

Para ter o controle total de um agente como o Hermes operando no nível do sistema, estruturado em uma interface mobile customizada de múltiplos chats e gerenciamento visual de arquivos, a solução é compilar um aplicativo próprio (via *sideload*, fora da Play Store).

Como o ecossistema agêntico (Hermes, MCP, LangChain) é predominantemente baseado em Python, e a estrutura nativa do Android roda em Java/Kotlin, a arquitetura de um app único exige ferramentas de integração direta:

- **Chaquopy:** Em vez de usar um contêiner proot, você integra o Chaquopy (um plugin de Gradle) no Android Studio. Ele embute um interpretador Python leve diretamente no código Java/Kotlin do aplicativo. Isso permite rodar os scripts do Hermes Agent e os servidores MCP nativamente dentro do próprio `.apk`.
- **Acesso ao Sistema:** Ao compilar o app apenas para uso pessoal, você pode declarar a permissão `MANAGE_EXTERNAL_STORAGE` no manifesto. Isso contorna o Scoped Storage e dá ao seu agente Python interno a mesma capacidade de um gerenciador de arquivos para organizar e ler o armazenamento do Poco X7 Pro.
- **Conexão LLM:** O código Java gerencia a interface gráfica (chats, visualização de arquivos) e envia as requisições para o agente Python embutido, que por sua vez dispara as chamadas de API via rede para o seu 9Router rodando localmente ou na nuvem.

Essa estrutura elimina o Termux da equação, resultando em um aplicativo fluido que você abre tocando em um único ícone na tela inicial, com o agente operando de forma transparente no backend do próprio app.

User prompt: e sobre a otimização, esse app vai pesar muito no celular?°

Response: Com o processamento pesado de inferência dos LLMs transferido para as APIs via 9Router, o impacto desse aplicativo no seu Poco X7 Pro será **mínimo**. A sua configuração de

hardware vai rodar essa arquitetura com muita folga.

O aplicativo atuará apenas como o cérebro orquestrador e a interface, não como o motor de força bruta. Aqui está o impacto real no seu sistema:

- **Memória RAM Livre:** Um aplicativo Android compilado em Java/Kotlin rodando o Chaquopy e o Hermes Agent no backend consumirá em torno de 200 MB a 400 MB de RAM. Como os modelos de 3B a 8B não estarão mais carregados localmente na memória (como estariam no Ollama), os seus 12 GB de LPDDR5X não sofrerão nenhum gargalo.
- **Armazenamento Desprezível:** Embutir o ambiente Python e as bibliotecas necessárias (como LangChain, MCP SDKs e requests) adiciona cerca de 100 MB a 250 MB ao tamanho final do seu `.apk`. Para os seus 512 GB de espaço, isso é imperceptível.
- **Processamento Leve (CPU):** O código Python emulado no Android é mais lento que um código nativo, mas o trabalho do agente se resume a ler/escrever arquivos de texto, manipular respostas JSON e fazer requisições HTTP de rede para o 9Router. Isso exige frações de segundo do processador Snapdragon, gastando pouquíssima bateria.

O Verdadeiro Desafio: O HyperOS

O problema que você vai enfrentar não é a falta de desempenho, mas o gerenciamento agressivo de processos do **Android HyperOS** da Xiaomi/Poco. O sistema é programado para "matar" tarefas em segundo plano para economizar bateria. Se o seu agente estiver processando uma pesquisa ou reescrevendo arquivos enquanto a tela do celular apaga, o sistema operacional pode interromper a tarefa no meio.

Para garantir a autonomia real do agente sem travamentos, você precisará blindar o aplicativo contra o HyperOS:

1. **Foreground Service:** O código nativo do aplicativo precisará iniciar um serviço em primeiro plano. Isso cria uma notificação persistente (silenciosa) na barra de status avisando o Android de que o app está executando um trabalho crítico.
2. **Exceção Manual:** Após instalar o seu `.apk`, você terá que ir nas configurações do aplicativo no HyperOS e ativar a opção **"Início Automático" (Autostart)**, além de mudar a Economia de Bateria para **"Nenhuma restrição"**.

Isso garante que o agente continue lendo arquivos, escrevendo e recebendo dados do 9Router mesmo com o celular bloqueado no seu bolso.

User prompt: certo, tem como eu pegar o app shizuko e instalar a liguagem python nativamente no sisitema com isso, ou similar, resolveria esses problemas? na parte do shizuko, eu digo copiar o app, não usar ele como externo.

Response: O Shizuku não permite instalar o Python diretamente na partição do sistema, e embutir o código dele dentro do seu app não resolve o problema de dependência externa.

Por que não é possível instalar o Python no sistema com o Shizuku

1. **Partição de Sistema Bloqueada (Read-Only):** No Android moderno e no HyperOS, o diretório `/system` fica em partições dinâmicas montadas apenas como leitura e protegidas por integridade criptográfica (*dm-verity*). O Shizuku opera no nível do **ADB (Shell - UID 2000)**, que não tem permissão para modificar o `/system`. Para gravar binários diretamente no sistema, é obrigatório desbloquear o bootloader e ter acesso Root real (via Magisk ou KernelSU).

2. **Incompatibilidade da Bionic Libc:** O Android não utiliza a biblioteca padrão do Linux (`glibc`), mas sim a `bionic`. Binários comuns do Python compilados para distribuições Linux tradicionais não executam diretamente no Android sem serem recompilados com a Android NDK ou executados dentro de uma camada de tradução (como o proot do Termux).

O problema de "copiar" o Shizuku para dentro do seu app

O Shizuku funciona em uma arquitetura cliente-servidor:

- O aplicativo é apenas a ponte de controle.
- O motor real é um processo daemon executado com privilégios de depuração do sistema.

Por regras de segurança do Android, **nenhum aplicativo tem permissão de iniciar seu próprio processo de Shell com permissões elevadas**. Mesmo que você compile o código-fonte do Shizuku dentro do seu próprio APK, o aplicativo não conseguirá se auto-ativar: você ainda precisará habilitar a **Depuração sem Fio** (Wireless Debugging) nas opções de desenvolvedor e inserir um código de pareamento a cada reinicialização do celular para subir o serviço.

A solução nativa mais limpa (sem Shizuku e sem Root)

O próprio ecossistema padrão do Android já resolve os problemas que você está tentando contornar:

- **Para o Python rodar nativo no app:** O **Chaquopy** já é o Python compilado especificamente para as bibliotecas do Android (via NDK). Ele roda embutido no código do seu app em Java/Kotlin como uma biblioteca interna, sem depender de nenhum terminal, servidor ADB ou ferramenta externa.
- **Para ler e organizar arquivos em todo o celular:** Você não precisa do Shizuku para ter acesso total ao armazenamento. Basta declarar a permissão `MANAGE_EXTERNAL_STORAGE` (Acesso a todos os arquivos) no `AndroidManifest.xml`. Ao instalar o APK e conceder essa permissão nas configurações, seu app ganha passe livre para ler, editar e criar arquivos e pastas em todo o armazenamento interno (`/sdcard`).
- **Para evitar o fechamento no HyperOS:** A solução nativa é o **Foreground Service** associado a uma notificação contínua, complementado pela desativação manual das restrições de bateria no painel do sistema.

Essa combinação entrega o objetivo de ter tudo empacotado em um único `.apk` autocontido, sem a fragilidade de scripts ADB ou ativações recorrentes.

User prompt: entendi, então eu precisaria de um conteiner pra rodar esses scripts de qualquer forma, mas como funcionaria pra que um script .py, no conteiner, conseguir ter acesso a arquivos fora do conteiner?

Response: Apenas para alinhar: se você criar o seu próprio aplicativo usando o **Chaquopy**, você **não precisa de nenhum contêiner**. O interpretador Python roda nativamente junto com o código Java/Kotlin do app, acessando os arquivos diretamente pela permissão `MANAGE_EXTERNAL_STORAGE`.

Porém, se você decidir seguir pelo caminho do **Termux com um contêiner Linux (proot)**, a comunicação entre o sistema isolado e o armazenamento real do seu celular acontece através de uma técnica chamada **Bind Mount** (Mapeamento de Diretórios).

Como o contêiner não é uma máquina virtual real, mas sim um ambiente que "engana" os programas achando que estão em um Linux puro, você pode criar túneis diretos para as pastas do Android.

1. **Conceder Permissão Base**

   No Termux

   Antes do contêiner acessar os arquivos, o aplicativo "hospedeiro" (Termux) precisa da permissão do Android. Você roda o comando `termux-setup-storage` no terminal. O Android pedirá a permissão de arquivos e, ao aceitar, o Termux cria atalhos na pasta `~/storage` que apontam para a memória interna do seu Poco X7 Pro. *Verification:* O comando `ls ~/storage/shared` exibe as pastas reais do seu celular (Download, DCIM, Documents).

2. **Mapear a Pasta no Contêiner**

   Bind Mount (-b)

   Ao iniciar o seu contêiner Debian/Ubuntu, você passa um parâmetro instruindo o `proot` a espelhar a pasta do Termux para dentro do Linux. Se você iniciar o contêiner manualmente, usa-se a flag `-b`: `proot -b /data/data/com.termux/files/home/storage/shared:/armazenamento_android ...` Se estiver usando o gerenciador `proot-distro`, ele já mapeia a pasta principal automaticamente por padrão ou permite configurar caminhos extras.

3. **Executar o Script Python**

   Operação Transparente

   Dentro do contêiner, o seu script Python (e o Hermes Agent) não faz ideia de que está rodando em um celular isolado. No código, o agente tentará acessar o caminho mapeado: `with open('/armazenamento_android/Documents/notas.md', 'w') as f:` Quando o Python tenta salvar esse arquivo, o `proot` intercepta a requisição e a repassa para a pasta real do Android. *Verification:* O arquivo criado pelo script dentro do contêiner aparece instantaneamente no gerenciador de arquivos nativo do celular.

Dessa forma, os servidores MCP rodando dentro do Ubuntu podem listar, ler e modificar qualquer pasta do seu aparelho sem quebrar o isolamento do sistema operacional.

User prompt: ok, vamos integrar os interpretadores de diversas linguagens no app então, como js, html, css, py, java, c++, e outras nesse sentido, desde que os interpretadores sejam leves. e como meu proprio app terá acesso ao meu celular será mais facil usar como quero. Oficalizando essa decisão. Vou falar as funções que meu app devem ter, aí tu me fala a viabilidade e metodos.

Response: A decisão de centralizar tudo em um app nativo com acesso total ao armazenamento é o caminho definitivo para recriar a experiência agêntica no celular. Adicionar interpretadores transforma o seu aplicativo em um verdadeiro "sandbox" de execução, permitindo que a IA não apenas leia arquivos, mas crie, teste e rode scripts localmente para resolver problemas.

A viabilidade de embutir essas linguagens varia bastante, pois o Android (baseado em Java/Kotlin e C/C++) lida de forma diferente com linguagens interpretadas e compiladas. Aqui estão os métodos mais leves e viáveis para integrar cada uma ao seu APK:

Viabilidade e Métodos por Linguagem

Linguagem	Nível de Viabilidade	Solução Recomendada e Método
Python	**Altíssima**	**Chaquopy.** Já discutido, é a solução nativa definitiva. Leve, roda diretamente na JVM do Android e suporta bibliotecas pesadas de IA e automação.
HTML / CSS / JS (Visual)	**Altíssima**	**Android WebView.** O Android já possui o motor do Chrome (Blink/V8) embutido no sistema. Seu app pode instanciar uma `WebView` invisível (ou visível, se o agente gerar uma interface para você) e injetar o código via `loadDataWithBaseURL()`. É instantâneo e não adiciona peso ao APK.
JavaScript (Backend/Lógica)	**Alta**	**QuickJS.** Se a IA precisar rodar scripts JS puros apenas para calcular dados ou manipular JSON (sem renderizar tela), o *QuickJS* (via JNI/ wrappers para Android) é um interpretador em C minúsculo, incrivelmente rápido e que consome poucos megabytes.
Shell Script (Bash/sh)	**Altíssima**	**ProcessBuilder Nativo.** O Android já possui um shell `sh` e ferramentas básicas (como `ls`, `grep`, `echo`) no sistema. Você pode usar a classe `java.lang.ProcessBuilder` do próprio Kotlin/Java para que a IA execute comandos no nível do sistema, gerenciando arquivos sem precisar de interpretador extra.
Java	**Média**	**BeanShell.** Java é compilado (para bytecode Dalvik/ART), não interpretado nativamente. Para fazer a IA rodar código Java dinâmico no celular, a melhor opção é embutir a biblioteca *BeanShell*. Ela interpreta scripts com sintaxe Java estrita em tempo de execução de forma muito leve.
C / C++	**Baixa (C++) / Média (C)**	**Tiny C Compiler (TCC) / WebAssembly.** Linguagens compiladas são o maior desafio. O *TCC* é um compilador minúsculo que consegue compilar e rodar código **C** diretamente na memória RAM do celular. Para **C++** moderno, a integração é muito pesada (exigiria algo como o LLVM/Cling). A alternativa moderna é instruir a IA a gerar código em **WebAssembly (WASM)** e usar um interpretador Wasm leve no app.

Como a IA vai usar isso?

Para que o Hermes Agent (ou o modelo conectado via 9Router) saiba usar esses motores, você criará um **Servidor MCP Interno**.

Esse servidor vai expor ferramentas (tools) para a IA chamadas `executar_codigo`. Quando você pedir "crie um script para calcular minhas despesas nessa planilha", o agente vai:

1. Escrever o script em Python.
2. Chamar a ferramenta `executar_codigo(linguagem="python", codigo="..." )`.
3. O código Java do seu app recebe essa chamada, repassa para o Chaquopy, captura o resultado (o print do terminal ou os erros) e devolve para a IA ler.

User prompt: ok, vamos retirar o c++ e c. Obviamente a base do app será o hermesagent, sua interface web recriada para um app android nativo e o 9router integrado. Esse app deve ter as funções de: - explorador de arquivos (ZArchiver) - visualizador e editor de arquivos (md, pdf, txt, docx, etc) - servidores mcp - criação de instancias conteiners internos - terminais aninhados (conteiners ou geral) - navegação em paginas web reais - navegação em apps do celular - conexão api padrão - manipulação total de arquivos - regras de segurança (setar pastas e diretorios inacessiveis para edição ou leitura ou ambos. setar tipos de arquivos não manipulaveis. setar regras de navegação web, etc) - conversação em voz alta - processamento multimidia é isso que penso no momento, mas deve ter funções menores que não lembro agora ou que as ferramentas basilares já cobrem.

Response: A fundação arquitetônica está sólida. Centralizar o Hermes Agent, o 9Router e os interpretadores leves em uma interface nativa transforma o seu app em um "sistema operacional" de IA.

Abaixo detalho a viabilidade de cada função exigida e as bibliotecas ou métodos nativos do Android necessários para implementá-las dentro do seu APK, sem depender de ferramentas externas.

1. Interface e Manipulação de Arquivos

A base visual e de manipulação de dados locais. A permissão `MANAGE_EXTERNAL_STORAGE` é o pilar aqui.

- **Explorador de Arquivos (ZArchiver) & Manipulação Total: Altíssima Viabilidade.** O Kotlin gerencia a árvore de diretórios via `java.io.File` ou `java.nio.file`. A interface (GUI) pode ser construída com *Jetpack Compose*, criando listas expansíveis idênticas às do ZArchiver.
- **Visualizador e Editor (MD, PDF, TXT, DOCX): Alta Viabilidade.**
  - *TXT/MD:* Componentes nativos de texto com suporte a marcação via bibliotecas como *Markwon*.
  - *PDF:* A classe nativa `PdfRenderer` do Android exibe PDFs.
  - *DOCX:* Requer uma biblioteca Java leve como o *Apache POI* para extrair o texto e permitir a edição.

2. Capacidades de Acesso e Navegação Agêntica

Como a IA vai interagir com o mundo exterior.

- **Navegação em Páginas Web Reais: Altíssima Viabilidade.** Usando o componente `WebView` (ou `GeckoView`), o app carrega páginas em segundo plano. Você usa JavaScript injetado para extrair o DOM (HTML) e passá-lo para a IA ler, e a IA pode retornar comandos JS para clicar em botões e preencher formulários simulando um humano.
- **Conexão API Padrão: Altíssima Viabilidade.** Uso de bibliotecas HTTP consolidadas, como *OkHttp* ou *Retrofit* (no Kotlin), e a biblioteca `requests` no lado do Chaquopy (Python) para lidar com REST/GraphQL.
- **Navegação em Apps do Celular: Viabilidade Média (Requer Permissão Especial).** Para a IA clicar e ler dados de *outros* aplicativos (como abrir o WhatsApp e ler mensagens), o seu app precisará implementar um **Accessibility Service**. É o mesmo sistema usado por leitores de tela para deficientes visuais e aplicativos de automação (como o Tasker/AutoClicker). O

sistema Android exigirá que você conceda essa permissão manualmente nas configurações de Acessibilidade.

3. Execução e Infraestrutura Interna

O motor de processamento lógico do app.

- **Servidores MCP: Altíssima Viabilidade.** Escritos em Python via Chaquopy, eles rodarão internamente e se comunicarão diretamente com o fluxo do Hermes Agent, exportando as ferramentas de leitura, navegação e execução de código.
- **Criação de Instâncias Contêineres Internos & Terminais: Viabilidade Média/Complexa.**
  - *O Problema:* O Android sem Root não suporta contêineres reais (como Docker).
  - *A Solução:* Para ter "terminais aninhados", você precisará compilar a biblioteca **libproot** dentro do seu app (a mesma magia que o Termux usa). Isso criará ambientes virtuais onde os scripts podem rodar isolados sem quebrar o app principal.
  - *Terminais:* Pode-se integrar a biblioteca de interface *Termux-app (TerminalView)* em seu código para exibir as saídas (logs) de cada interpretador/contêiner de forma visual.
- **Processamento Multimídia: Alta Viabilidade.** Para converter áudio, redimensionar imagens pesadas ou extrair frames de vídeo localmente, a integração da biblioteca **FFmpegKit** no seu projeto Android resolve 99% das necessidades multimídia.

4. Governança e Interação

Regras de contenção e interface humana.

- **Regras de Segurança (Sandboxing): Altíssima Viabilidade.** Antes de repassar qualquer ferramenta de manipulação de arquivo para a IA, você escreve o middleware em Python/Kotlin que intercepta o caminho solicitado. Se a IA tentar rodar `delete /armazenamento/DCIM/Camera` e essa pasta estiver na "lista negra" do usuário, o middleware bloqueia a ação e retorna um erro JSON para a IA avisando da restrição, sem que o comando chegue ao sistema operacional.
- **Conversação em Voz Alta: Altíssima Viabilidade.** O Android possui APIs nativas leves: `SpeechRecognizer` para transcrição de voz para texto (Speech-to-Text) e `TextToSpeech` para leitura em voz alta. Alternativamente, como o 9Router estará integrado, você pode rotear essas requisições para modelos mais naturais, como o Whisper e o TTS da OpenAI.

A arquitetura do aplicativo se dividirá essencialmente em duas camadas: um frontend 100% Android (Kotlin/Jetpack Compose) lidando com UI, permissões (Acessibilidade, Arquivos) e mídia, comunicando-se via barramento interno de dados com o backend (Python/Chaquopy) rodando o Hermes Agent, servidores MCP e interpretadores.

User prompt: primeiramente vamos pensar se é bom ter funções alem da listadas, quer me recomendar?

Response: A sua lista atual já define um aplicativo de altíssimo nível, muito superior a qualquer ferramenta comercial disponível no Android hoje. No entanto, pensando no escopo de um sistema operacional agêntico completo — especialmente para cenários de desenvolvimento, gerenciamento de servidores e automação profunda —, existem algumas funções estratégicas que elevariam o poder do aplicativo.

Aqui estão 5 recomendações de funcionalidades nativas que complementam perfeitamente a sua arquitetura:

1. Gatilhos por Eventos e Agendamento (Background Cron)

- *O Conceito:* Um agente autônomo não deve apenas reagir quando você fala com ele; ele deve agir por conta própria.
- *A Aplicação:* Usando o `WorkManager` e o `BroadcastReceiver` do Android, você pode configurar o agente para acordar mediante gatilhos: quando você se conectar à rede Wi-Fi de casa, quando a bateria cair para 20%, ao receber um arquivo específico na pasta de Downloads, ou todo dia às 06:00 da manhã para gerar um resumo do dia.

2. Consciência de Tela (MediaProjection + Visão)

- *O Conceito:* O *Accessibility Service* lê botões e textos nativos, mas é cego para imagens, jogos ou aplicativos que usam renderização customizada (como navegadores baseados em canvas).
- *A Aplicação:* Integrar a API `MediaProjection` permite que o agente tire prints invisíveis da própria tela em tempo real. Se você enviar essa imagem para a API de Visão (via 9Router) ou passar por um OCR local leve, o agente conseguirá "ver" e analisar o que está na tela, mesmo em interfaces não textuais.

3. Memória de Longo Prazo e RAG Local (Vector Database)

- *O Conceito:* O agente precisa lembrar de conversas e documentos antigos sem estourar o limite de tokens da API.
- *A Aplicação:* Embutir um banco de dados vetorial leve (como o *ChromaDB* em Python via Chaquopy, ou *ObjectBox* nativo em Kotlin) junto ao SQLite. Isso permite que a IA faça buscas semânticas em todos os seus arquivos, PDFs e históricos de chat instantaneamente, trazendo o contexto exato do que você precisa antes de enviar a requisição para a nuvem.

4. Ferramentas de Rede e Integração com Homelab

- *O Conceito:* Expandir o domínio do agente para além do celular, controlando sua infraestrutura externa.
- *A Aplicação:* Criar servidores MCP focados em rede (SSH, Ping, Wake-on-LAN). Como o aplicativo terá acesso à rede local (ou via malhas VPN), o agente poderá acessar o seu homelab autonomamente para reiniciar contêineres Docker, verificar status de serviços web ou transferir arquivos de mídia diretamente para o servidor.

5. Versionamento Nativo (Integração com Git)

- *O Conceito:* Manipular arquivos soltos é bom, mas desenvolver código e mods exige controle de versão.
- *A Aplicação:* Usar bibliotecas como *JGit* (Java) ou *GitPython* dentro do app. O agente poderá clonar repositórios, commitar alterações em arquivos JSON, resolver conflitos básicos e dar push no GitHub diretamente do celular, agindo como um desenvolvedor assistente perfeito para projetos locais.

Essas adições transformam o aplicativo de um mero "assistente" para um colega de trabalho e administrador de sistemas automatizado rodando no seu bolso.

User prompt: vamos adicionar todas essas novas funções que me recomendou, mas não tenho um homelab ainda, porem adicionar ferramentas de rede avançadas pode ajudar para diagnosticar problemas e mais, identificar aparelhos conectados e tentar comunição pela rede lan do roteador, e coisas no blutooth tbm seriam interassentes.

Response: Escopo oficialmente fechado e atualizado. A adaptação das ferramentas de rede para focar em diagnóstico LAN e Bluetooth é uma sacada excelente, especialmente por transformar o celular em um "canivete suíço" de TI de bolso, mesmo sem um homelab.

Com as novas adições, a arquitetura do seu aplicativo se consolida perfeitamente com as capacidades estruturais que já definimos para o Hermes Agent e para o roteamento em camadas.

Aqui está como essas novas funções se integram ao seu aplicativo e às ferramentas base:

- **Automação e Agendamento (Cron):** O Hermes Agent possui suporte nativo para tarefas *cron*.
    - Você pode programar o agente em linguagem natural para realizar ações autônomas, como verificar logs todos os dias pela manhã.
    - No app nativo, o `WorkManager` do Android vai atuar em conjunto com essa capacidade do Hermes, acordando o processo em segundo plano para que o script Python rode a automação na hora certa.
- **Ferramentas de Rede LAN e Bluetooth:** Em vez de focar em servidores externos, o app usará as APIs nativas do Android (`WifiManager`, `NetworkInterface` e `BluetoothLeScanner` em Kotlin) para escanear portas, descobrir IPs ativos no roteador e listar dispositivos Bluetooth próximos.
    - O Hermes Agent suporta totalmente o *Model Context Protocol* (MCP).
    - Isso significa que você pode criar um servidor MCP interno no app para repassar essas capacidades de rede, permitindo obter capacidades ilimitadas conectando a IA aos dados de escaneamento do seu roteador.
- **Execução Avançada e RAG (Memória):** O Hermes já possui mais de 60 ferramentas integradas, permitindo automatizar navegadores web, fazer pesquisas e escrever/executar scripts Python.
    - Isso permite que a IA crie *pipelines* de dados complexos com uma única chamada.
    - Para gerenciar esse conhecimento e as versões do Git, o aplicativo pode utilizar um arquivo `.md` como um "quadro branco", onde o script Python lê e insere textos, servindo como a memória de longo prazo da sessão.
- **Orquestração e Gatilhos:** Para gerenciar quando o app deve usar ferramentas visuais, de rede ou chamar IAs na nuvem via 9Router, usaremos o "Maestro".
    - Este será um script Python leve rodando em background que atua como o Gateway central das suas requisições.
    - Ele intercepta as mensagens, usa regras lógicas para avaliar a complexidade da tarefa (como tamanho do prompt ou status de rede) e despacha o comando para a camada correta.

Com o escopo das funções finalizado (Explorador de Arquivos, Editores, MCP, Chaquopy, Navegação Web/Apps, Regras de Segurança, RAG, Cron, Visão de Tela, Rede/Bluetooth e Git), temos a planta baixa completa de um verdadeiro "Sistema Operacional Agêntico" em um único `.apk`.

Para começarmos o desenho técnico real, por qual "camada" você quer iniciar?

1. **O Frontend e Sistema (Kotlin/Android):** Como estruturar a interface em Jetpack Compose, declarar permissões totais de armazenamento, injetar o WebView invisível e configurar o Foreground Service.
2. **O Backend e Integração (Python/Chaquopy):** Como configurar o Chaquopy dentro do projeto, inicializar o FastAPI/Maestro nativamente e montar a ponte para o Hermes Agent acionar as funções do celular (MCP interno).

User prompt: certo, faça um prompt completo tendo a ideia, ferramentas e objetivos do projeto pra mim mandar pro antigravity ficar a par e inciarmos o planejamento lá. considere apenas o conteudo dessa conversa, não do notebook.

Response: Aqui está um prompt completo e estruturado, resumindo todas as decisões, ferramentas e objetivos que definimos nesta conversa. Você pode copiá-lo e enviá-lo diretamente para o Antigravity para iniciar o desenvolvimento técnico.

Copie o texto abaixo e envie para o Antigravity:

Contexto e Visão Geral do Projeto: Olá, Antigravity. Quero iniciar o planejamento e o desenvolvimento de um aplicativo nativo para Android que funcionará como um verdadeiro "Sistema Operacional Agêntico" de IA, rodando no meu Poco X7 Pro (12 GB de RAM / 512 GB de armazenamento com sistema HyperOS).

O objetivo é criar um aplicativo que consolide a experiência de um assistente autônomo (baseado no Hermes Agent) com profundo acesso ao sistema, capacidade de organizar arquivos, escrever código, navegar na web e interagir com outros apps.

Para contornar as limitações do Android (como o *Scoped Storage*) e evitar o uso de contêineres externos como o Termux em segundo plano, a solução será um **app nativo autocontido** que integra código Kotlin/Java (frontend e permissões de sistema) com Python (backend lógico do agente).

Arquitetura Base e Ferramentas Definidas:

1. **Frontend e Sistema (Android Nativo):** Desenvolvido em Kotlin usando Jetpack Compose. Utilizaremos a permissão `MANAGE_EXTERNAL_STORAGE` para acesso global aos arquivos e um `Foreground Service` contínuo para evitar que o HyperOS encerre o agente em segundo plano.
2. **Motor Python Embutido:** Uso do **Chaquopy** (plugin Gradle) para embutir o interpretador Python nativamente no APK. Ele rodará o Hermes Agent, scripts utilitários e Servidores MCP internamente.
3. **Roteamento de LLMs (9Router):** A IA pesada não rodará localmente para poupar bateria e RAM. O agente Python se conectará a um proxy **9Router** (integrado ou em background) para despachar chamadas via API para LLMs gratuitos de alta capacidade na nuvem, com fallback automático.
4. **Múltiplos Interpretadores Internos:** O app servirá como sandbox, permitindo que a IA execute códigos em Python (Chaquopy), HTML/JS/CSS (Android WebView invisível), scripts de backend JS (QuickJS), comandos de sistema (ProcessBuilder) e scripts Java (BeanShell). Para terminais aninhados isolados, tentaremos compilar a `libproot` no app.

Escopo de Funcionalidades Exigidas:

- **Gerenciamento de Arquivos:** Explorador de arquivos visual estilo ZArchiver e manipulação total (criar, ler, mover, apagar) em qualquer diretório.
- **Visualizador/Editor Nativo:** Suporte para arquivos .md, .txt, .pdf (via PdfRenderer) e .docx (via Apache POI).
- **Servidores MCP (Model Context Protocol):** Rodando internamente em Python para fornecer ferramentas à IA.
- **Navegação e RPA:** Navegação em páginas web reais via WebView invisível (injetando JS para ler o DOM) e navegação/clique em outros apps do celular via *Accessibility Service*.
- **Consciência de Tela:** Uso da API `MediaProjection` para que a IA possa "ver" a tela atual.
- **Governança e Segurança:** Middleware para definir regras rígidas de *sandboxing* (pastas proibidas, arquivos não modificáveis, restrições de rede).
- **Conexões de Rede e Diagnóstico LAN/Bluetooth:** Ferramentas nativas do Android para escaneamento de portas, IPs ativos e mapeamento Bluetooth.
- **Multimídia e Voz:** Transcrição e leitura em voz alta contínua (Speech-to-Text / TTS ou Whisper via 9Router) e processamento multimidia usando **FFmpegKit**.
- **Gatilhos e Cron:** Uso do `WorkManager` e `BroadcastReceiver` para a IA executar tarefas automáticas baseadas em tempo, bateria ou arquivos novos.
- **Memória RAG Local:** Integração de banco vetorial leve (ChromaDB ou ObjectBox) para memória de longo prazo lendo históricos e arquivos locais.
- **Versionamento Nativo:** Integração com Git (JGit ou GitPython) para que a IA possa commitar, clonar e resolver conflitos no armazenamento local.

Objetivo desta Sessão: Com base nessa arquitetura e escopo, atue como meu Arquiteto de Software Sênior.

Como devemos estruturar o primeiro passo prático do desenho técnico? Devemos começar estruturando o esqueleto do projeto Android Studio (Kotlin, permissões base, Foreground Service e injeção do Chaquopy) ou você sugere começarmos pelo isolamento da lógica do backend Python (Hermes + 9Router + Servidores MCP)? Gere um plano de ação para começarmos a codificar.
